RELATÓRIO DE CÓDIGO SELECIONADO

Seleção técnica parcial, não é o fonte integral.

```
# Título: Alerta de Alagamentos
# Participantes: identificação omitida conforme solicitado.
# Referência: 028b704
```

Produção: 2.096 linhas físicas em 21 arquivos; inclui comentários e linhas vazias. As páginas seguintes selecionam 135 linhas existentes. Testes, configurações e arquivos gerados ficam separados.

Tabela 1 – Inventário de linhas físicas

Arquivo de produção	**Linhas**
backend/app.py	315
backend/autenticacao.py	23
backend/database.py	43
backend/init_db.py	17
backend/midia.py	169
backend/schema.sql	39
mobile/App.js	192
mobile/index.js	8
mobile/src/components/AnexoViewer.js	109
mobile/src/components/Interface.js	76
mobile/src/components/LocalizacaoCampos.js	77
mobile/src/components/MapaOcorrencias.native.js	68
mobile/src/components/MapaOcorrencias.web.js	107
mobile/src/components/SeletorAnexo.js	85
mobile/src/config.js	30
mobile/src/screens/AuthScreen.js	115
mobile/src/screens/CadastroScreen.js	123
mobile/src/screens/DetalhesScreen.js	134
mobile/src/screens/ListaScreen.js	155
mobile/src/services/api.js	162
mobile/src/theme.js	49

Fonte: contagem dos arquivos do commit 028b704 (2026).

Confirmar a aceitação de seleção legível + fonte completo separado. > indica quebra visual, sem alterar a linha original.

1.1 Trechos selecionados – parte 1

Foco a explicar pelo grupo: Conexão, chave estrangeira, commit, rollback e fechamento

Quadro 1 – Trecho de backend/database.py

```
 13 | def conectar(database):
 14 |     """Cada operação recebe sua conexão e confirma ou desfaz a
    > transação."""
 15 |     conexao = sqlite3.connect(str(database), timeout=10)
 16 |     conexao.row_factory = sqlite3.Row
 17 |     conexao.execute("PRAGMA foreign_keys = ON")
 18 |     try:
 19 |         yield conexao
 20 |         conexao.commit()
 21 |     except Exception:
 22 |         conexao.rollback()
 23 |         raise
 24 |     finally:
 25 |         conexao.close()
```

Fonte: código existente do projeto (028b704), backend/database.py, linhas 13–25.
A conexão é configurada com acesso por nome de coluna e chaves estrangeiras ativas. O bloco confirma a transação no sucesso, desfaz em exceção e fecha o recurso ao terminar.

Foco a explicar pelo grupo: SHA-256 do token e criação de sessão com validade

Quadro 2 – Trecho de backend/autenticacao.py

```
 12 | def hash_token(token):
 13 |     return hashlib.sha256(token.encode("utf-8")).hexdigest()
 14 |
 15 |
 16 | def criar_sessao(conexao, usuario_id):
 17 |     token = secrets.token_urlsafe(32)
 18 |     expira_em = (datetime.now(timezone.utc) +
    > timedelta(days=7)).isoformat(timespec="seconds")
 19 |     conexao.execute(
 20 |         "INSERT INTO sessoes(usuario_id, token_hash, expira_em)
    > VALUES (?, ?, ?)",
 21 |         (usuario_id, hash_token(token), expira_em),
 22 |     )
 23 |     return token, expira_em
```

Fonte: código existente do projeto (028b704), backend/autenticacao.py, linhas 12–23.
O token aleatório é entregue ao cliente, enquanto apenas seu hash é registrado. A validade de sete dias limita a sessão, e a consulta posterior usa o hash para identificar a conta.

1.2 Trechos selecionados – parte 2

Foco a explicar pelo grupo: Autenticação Bearer antes da operação

Quadro 3 – Trecho de backend/app.py

```
 49 |         partes = request.headers.get("Authorization", "").split()
 50 |         if len(partes) != 2 or partes[0].lower() != "bearer":
 51 |             return falha("Entre na sua conta para continuar.", 401)
 52 |         with conectar(app.config["DATABASE"]) as conexao:
 53 |             usuario = conexao.execute(
 54 |                 """SELECT u.id, u.nome, u.email FROM sessoes s
 55 |                    JOIN usuarios u ON u.id = s.usuario_id
 56 |                    WHERE s.token_hash = ? AND s.expira_em > ?""",
 57 |                 (hash_token(partes[1]), agora_utc()),
 58 |             ).fetchone()
 59 |         if usuario is None:
 60 |             return falha("Sessão inválida ou expirada. Entre
    > novamente.", 401)
 61 |         g.usuario = dict(usuario)
 62 |         g.token_hash = hash_token(partes[1])
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 49–62.

A requisição protegida precisa apresentar Bearer válido. A associação entre sessão e conta é consultada antes de disponibilizar a operação; uma sessão ausente ou vencida recebe 401.

Foco a explicar pelo grupo: Validação do par latitude e longitude

Quadro 4 – Trecho de backend/app.py

```
217 |         latitude, longitude = dados.get("latitude"),
    > dados.get("longitude")
218 |         if latitude is None and longitude is None:
219 |             valores.update(latitude=None, longitude=None)
220 |             return valores, None
221 |         for campo, valor, limite in (("latitude", latitude, 90),
    > ("longitude", longitude, 180)):
222 |             if isinstance(valor, bool) or (not multipart and not
    > isinstance(valor, (int, float))):
223 |                 return None, falha("Informe latitude e longitude
    > numéricas juntas.", 400)
224 |             try:
225 |                 numero = float(valor)
226 |             except (TypeError, ValueError, OverflowError):
227 |                 return None, falha("Informe latitude e longitude
    > numéricas juntas.", 400)
228 |             if not math.isfinite(numero) or not -limite <= numero <=
    >  limite:
229 |                 return None, falha(f"A {campo} deve estar entre
    > {-limite} e {limite}.", 400)
230 |             valores[campo] = numero
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 217–230.

Latitude e longitude são aceitas em conjunto. A validação rejeita tipos indevidos, valores não finitos e coordenadas fora dos intervalos, evitando posições inválidas no mapa.

1.3 Trechos selecionados – parte 3

Foco a explicar pelo grupo: INSERT parametrizado e remoção de arquivo se a transação falhar

Quadro 5 – Trecho de backend/app.py

```
250 |         try:
251 |             # Falha no INSERT, no anexo ou no commit desfaz o banco
    > e remove o arquivo.
252 |             with conectar(app.config["DATABASE"]) as conexao:
253 |                 cursor = conexao.execute(
254 |                     """INSERT INTO ocorrencias
255 |                        (bairro, endereco, descricao, gravidade,
    > situacao, criado_em, autor_id, latitude, longitude)
256 |                        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
257 |                     (valores["bairro"], valores["endereco"],
    > valores["descricao"], valores["gravidade"],
258 |                      "ativo", agora_utc(), g.usuario["id"],
    > valores["latitude"], valores["longitude"]),
259 |                 )
260 |                 identificador = cursor.lastrowid
261 |                 if anexo:
262 |                     conexao.execute(
263 |                         """INSERT INTO anexos(ocorrencia_id,
    > nome_arquivo, tipo, mime_type, tamanho)
264 |                            VALUES (?, ?, ?, ?, ?)""",
265 |                         (identificador, anexo["nome_arquivo"],
    > anexo["tipo"], anexo["mime_type"], anexo["tamanho"]),
266 |                     )
267 |                 registro = conexao.execute("SELECT * FROM
    > ocorrencias WHERE id = ?", (identificador,)).fetchone()
268 |                 resultado = serializar_ocorrencia(conexao, registro)
269 |         except Exception:
270 |             if anexo:
271 |                 apagar_arquivo(app.config["UPLOAD_FOLDER"],
    > anexo["nome_arquivo"])
272 |             raise
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 250–272.
Os valores são passados separadamente ao comando SQL. A ocorrência e seus metadados são confirmados na transação; a exceção também remove o arquivo preparado.

1.4 Trechos selecionados – parte 4

Foco a explicar pelo grupo: Autorização de alteração por autoria

Quadro 6 – Trecho de backend/app.py

```
292 |         with conectar(app.config["DATABASE"]) as conexao:
293 |             registro = conexao.execute("SELECT * FROM ocorrencias
    > WHERE id = ?", (id,)).fetchone()
294 |             if registro is None:
295 |                 return falha("Ocorrência não encontrada.", 404)
296 |             if not pode_alterar(registro):
297 |                 return falha("Somente quem cadastrou pode alterar
    > esta ocorrência.", 403)
298 |             conexao.execute("UPDATE ocorrencias SET situacao = ?
    > WHERE id = ?", (situacao.strip(), id))
299 |             registro = conexao.execute("SELECT * FROM ocorrencias
    > WHERE id = ?", (id,)).fetchone()
300 |             resultado = serializar_ocorrencia(conexao, registro)
301 |         return jsonify(resultado), 200
```

Fonte: código existente do projeto (028b704), backend/app.py, linhas 292–301.

A autorização compara a conta da sessão com o autor antes do UPDATE. Outra conta pode consultar o registro compartilhado, mas recebe 403 ao tentar resolver a ocorrência.

Foco a explicar pelo grupo: Orientação e recriação de imagem sem EXIF

Quadro 7 – Trecho de backend/midia.py

```
 38 |             with Image.open(BytesIO(conteudo)) as foto:
 39 |                 foto.load()
 40 |                 with ImageOps.exif_transpose(foto) as orientada:
 41 |                     modo = "RGB" if formato == "JPEG" else "RGBA"
 42 |                     with orientada.convert(modo) as convertida:
 43 |                         # Uma imagem nova elimina EXIF, GPS e os
    > demais metadados.
 44 |                         with Image.new(modo, convertida.size) as
    > limpa:
 45 |                             limpa.paste(convertida)
 46 |                             saida = BytesIO()
 47 |                             limpa.save(saida, format=formato,
    > **({"quality": 85} if formato == "JPEG" else {}))
 48 |                             resultado = saida.getvalue()
```

Fonte: código existente do projeto (028b704), backend/midia.py, linhas 38–48.

A imagem é orientada e recriada a partir dos pixels. O arquivo resultante não conserva os metadados EXIF da imagem de entrada, incluindo sua possível localização.

1.5 Trechos selecionados – parte 5

Foco a explicar pelo grupo: Fetch com Bearer, AbortController e multipart

Quadro 8 – Trecho de mobile/src/services/api.js

```
 49 |     resposta = await fetch(`${API_URL}${caminho}`, {
 50 |       ...opcoes,
 51 |       headers: {
 52 |         Accept: arquivo ? '*/*' : 'application/json',
 53 |         ...(!publica && token ? { Authorization: `Bearer ${token}` }
    >  : {}),
 54 |         ...(opcoes.body && !formulario ? { 'Content-Type':
    > 'application/json' } : {}),
 55 |       },
 56 |       signal: controlador.signal,
 57 |     });
 58 |     // No multipart, o próprio cliente define Content-Type com seu
    > boundary.
 59 |     if (arquivo && resposta.ok) return await resposta.blob();
```

Fonte: código existente do projeto (028b704), mobile/src/services/api.js, linhas 49–59.
A função centraliza o envio da sessão e o cancelamento por prazo. No multipart, o cliente define o boundary; a resposta precisa ser verificada antes de indicar sucesso.

Foco a explicar pelo grupo: Permissão em primeiro plano e GPS com prazo

Quadro 9 – Trecho de mobile/src/components/LocalizacaoCampos.js

```
 27 |       // A permissão só é solicitada após o toque. Não há
    > rastreamento em segundo plano.
 28 |       const permissao = await
    > Location.requestForegroundPermissionsAsync();
 29 |       if (!permissao.granted) {
 30 |         throw new Error('Localização não autorizada. Você pode
    > continuar sem GPS e informar o endereço.');
 31 |       }
 32 |       if (!montada.current || bloqueado.current) return;
 33 |       /** @type {Promise<never>} */
 34 |       const prazo = new Promise((_, rejeitar) => {
 35 |         temporizador = setTimeout(() => rejeitar(new Error('O GPS
    > demorou para responder. Tente novamente ou continue sem
    > localização.')), 15000);
 36 |       });
 37 |       const local = await Promise.race([
 38 |         Location.getCurrentPositionAsync({ accuracy:
    > Location.Accuracy.Balanced }), prazo,
 39 |       ]);
 40 |       const { latitude, longitude } = local.coords;
 41 |       if (!Number.isFinite(latitude) || !Number.isFinite(longitude))
    >  {
 42 |         throw new Error('O dispositivo não informou uma localização
    > válida.');
 43 |       }
 44 |       // Uma resposta atrasada não pode alterar um formulário que já
    >  foi enviado.
 45 |       if (montada.current && !bloqueado.current) onChange({
    > latitude, longitude });
```

Fonte: código existente do projeto (028b704), mobile/src/components/LocalizacaoCampos.js, linhas 27–45.
A permissão é solicitada por ação do participante e a espera tem prazo. O resultado não deve alterar uma tela desmontada ou um formulário que já esteja bloqueado para envio.

1.6 Trechos selecionados – parte 6

Foco a explicar pelo grupo: Marcadores nativos com coordenadas dos relatos

Quadro 10 – Trecho de mobile/src/components/MapaOcorrencias.native.js

```
 42 |           {pontos.map((item) => (
 43 |             <Marker
 44 |               key={item.id} coordinate={{ latitude: item.latitude,
    > longitude: item.longitude }}
 45 |               title={item.bairro} description={`Gravidade relatada:
    > ${rotuloGravidade(item.gravidade)}`}
 46 |               pinColor={item.gravidade === 'alta' ? '#A73535' :
    > item.gravidade === 'media' ? '#B2750B' : cores.verde}
 47 |               onPress={() => onAbrirDetalhes?.(item.id)}
 48 |             />
 49 |           ))}
```

Fonte: código existente do projeto (028b704), mobile/src/components/MapaOcorrencias.native.js, linhas 42–49.

O mapa apresenta somente as coordenadas existentes nos relatos. O marcador usa os dados retornados pela API e oferece acesso ao registro correspondente.